

TRABAJO TERMINAL II

FlowCode: Compilador de Diagramas de Flujo a Código C para Dispositivos Móviles

Número del TT: 2026-A038

Grupo: 8CM1

Entregable 3 Ciclo 6: Integración y Pruebas

Presenta:

García Quiroz Gustavo Ivan

Profesor de seguimiento del TT:

Prof. Edgardo Adrián Franco Martínez

Introducción a la Integración y Pruebas (Ciclo 6)

Este ciclo materializa la consolidación del compilador FlowCode, uniendo las fases individuales de los ciclos anteriores en un canal de conversión (pipeline) completo y funcional. Además de la integración visual y de procesos, se establecieron metodologías de pruebas rigurosas para validar tanto los módulos aislados como el comportamiento del sistema de extremo a extremo.

1. Orquestación del Flujo de Conversión (Pipeline)

Se implementó el componente central que coordina la ejecución secuencial del análisis léxico, sintáctico, semántico, la optimización y la generación final de código C. Este pipeline asegura la propagación coherente de los datos entre fases, garantizando que la Tabla de Símbolos y el AST permanezcan sincronizados desde la interpretación del lienzo visual hasta la emisión del texto final.

2. Interfaz de Resultados y Sistema de Diagnóstico

Se desarrolló el componente visual para presentar al usuario los resultados de la compilación. El diálogo interactivo fue dotado de pestañas diferenciadas para exponer las métricas, las unidades léxicas, el AST y el código C con resaltado de sintaxis.

Simultáneamente, se consolidó el sistema de visualización de errores, incorporando una codificación por color según la severidad (rojo oscuro para errores fatales, naranja para advertencias, etc.), lo cual facilita la depuración y fomenta el autoaprendizaje del estudiante al diseñar sus algoritmos.

3. Pruebas Unitarias del Motor de Conversión

Se estructuró y ejecutó un conjunto de pruebas automatizadas aisladas para garantizar la fiabilidad técnica de cada módulo interno (lexer, parser, validadores semánticos y optimizador).

Mediante casos predefinidos (positivos y negativos), se verificó el correcto reconocimiento de sentencias, el balance de expresiones lógicas y la prevención de fallos críticos. Estas pruebas demostraron la robustez del motor frente a entradas malformadas, evitando cierres inesperados de la aplicación.

4. Pruebas de Integración de Extremo a Extremo (E2E)

Adicional a las pruebas unitarias, se desarrollaron escenarios automatizados de tipo Extremo a Extremo (End-to-End). Estas pruebas emulan el flujo del usuario desde la inicialización de nodos en el lienzo gráfico hasta la verificación sintáctica y estructural del programa C resultante en el estándar C99.

La aprobación de estas pruebas constata que las cinco fases del conversor interactúan sin pérdida de información semántica, cubriendo adecuadamente la conversión de los símbolos requeridos del estándar ISO 5807.

5. Pruebas Funcionales y Mapeo de Casos de Uso

Finalmente, se llevaron a cabo pruebas manuales sobre el dispositivo físico destino (p. ej. Samsung Galaxy A26 5G). Esto permitió validar funcionalidades que dependen de componentes externos y de la interfaz de usuario en tiempo de ejecución.

Entre estas validaciones se verificó el registro de cuentas de usuario, la persistencia local de los proyectos en carpetas y la sincronización de archivos hacia la nube mediante Firebase. Los resultados exitosos trazaron la cobertura completa de los casos de uso documentados en la especificación del sistema.

Evidencias de Integración y Pruebas Automatizadas

A continuación, se ilustran escenarios integrales de validación sobre el compilador FlowCode, contrastando la estructura diseñada en la aplicación con los reportes generados y las consolas de prueba.

PROBLEMS20+OUTPUTDEBUG CONSOLETERMINALPORTSTEST RESULTSGITLENSAZURE

env) PS C:\Users\ivan-\Documents\Github\Trabajo-Terminal\Flowdiagramapp>

FLOWCODE COMPILER PERFORMANCE BENCHMARK REPORT
Trabajo Terminal 2026-A038

RESUMEN DE RESULTADOS

✓ Linear-10	10 nodos	47.6 ms
✓ Linear-25	25 nodos	10.6 ms
✓ NestedConditional-25	25 nodos	7.6 ms
✓ LoopDiagram-25	25 nodos	9.6 ms
✓ IODiagram-25	25 nodos	8.4 ms
✓ Linear-50	50 nodos	18.8 ms
✓ MixedComplex-50	50 nodos	7.6 ms
✓ Optlevel-0	50 nodos	5.4 ms
✓ Optlevel-1	50 nodos	4.8 ms
✓ Optlevel-2	50 nodos	6.6 ms
✓ MediumComplexity-Validation	50 nodos	7.0 ms
✓ Linear-75	75 nodos	16.0 ms
✓ Linear-100	100 nodos	15.0 ms
✓ LargeDiagram-100	100 nodos	9.2 ms

DISTRIBUCIÓN DE TIEMPO POR FASE (LargeDiagram-100)

Léxico	13.5%
Sintáctico	37.8%
Semántico	10.8%
Optimización	18.9%
Generación	18.9%

MÉTRICAS DE RENDIMIENTO

- Promedio nodos/segundo: 5521.09
- Total benchmarks ejecutados: 14
- Todos exitosos: SI ✓

PROBLEMS20+OUTPUTDEBUG CONSOLETERMINALPORTSTEST RESULTSGITLENSAZURE

env) PS C:\Users\ivan-\Documents\Github\Trabajo-Terminal\Flowdiagramapp>

00:27 +151 -1: C:\Users\ivan-\Documents\Github\Trabajo-Terminal\Flowdiagramapp\test\compiler\compiler_integration_test.dart: MET-01: Compilation Metrics MET-01.2: Report generation works

✓ MET-01.2: Report generated successfully

00:33 +211 -2: C:\Users\ivan-\Documents\Github\Trabajo-Terminal\Flowdiagramapp\test\compiler\semantic_analyzer_test.dart: DiagramSemanticAnalyzer - Data Nodes (I/O) Detect undeclared variable in input [E]

Expected: <true>

Actual: <false>

package:matcher expect

package:flutter_test/src/widget_tester.dart 474:18 expect

test\compiler\semantic_analyzer_test.dart 463:7 main.<fn>.<fn>

To run this test again: C:\Flutter\bin\cache\dart-sdk\bin\dart.exe test C:\Users\ivan-\Documents\Github\Trabajo-Terminal\Flowdiagramapp\test\compiler\semantic_analyzer_test.dart -p vm --plain-name "DiagramSemanticAnalyzer - Data Nodes (I/O) Detect undeclared variable in input"

00:36 +325 -2: Some tests failed.

(.venv) PS C:\Users\ivan-\Documents\Github\Trabajo-Terminal\Flowdiagramapp>

1KBuildJava: ReadyLn 445, Col 1 (26 selected)Spaces: 2UTF-8CRLFMarkdownSM A266M (android-arm64)Prettier

RESULTADO GENERAL DE TEST RUNNER